

HW8

Sri Srinivasan

September 2025

Problem 1

(a)

$$W^{(\alpha)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W^{(\beta)} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad X = \begin{bmatrix} -2.17 & 1.98 & 2.41 & -2.03 \\ 0.02 & -0.01 & 0.01 & -0.02 \end{bmatrix}.$$

Encoder $W^{(\alpha)}$. Since $W^{(\alpha)} = I$, we have $W^{(\alpha)}X = X$, hence $\|X - W^{(\alpha)}X\|_F^2 = 0$. Also, $\|W^{(\alpha)}\|_F^2 = \|I\|_F^2 = \text{tr}(I) = m = 2$.

Encoder $W^{(\beta)}$. Left-multiplying X by $W^{(\beta)} = \text{diag}(1, 0)$ keeps the first row and zeros the second, and

$$W^{(\beta)}X = \begin{bmatrix} -2.17 & 1.98 & 2.41 & -2.03 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

So,

$$X - W^{(\beta)}X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.02 & -0.01 & 0.01 & -0.02 \end{bmatrix}, \quad \|X - W^{(\beta)}X\|_F^2 = 0.02^2 + (-0.01)^2 + 0.01^2 + (-0.02)^2 = 0.001.$$

Also, $\|W^{(\beta)}\|_F^2 = 1$.

table.

Encoder	Reconstruction Loss	Regularization Loss
α	0	2
β	0.001	1

(b)

Let the SVD of X be $X = U\Sigma V^\top$ with singular values $\sigma_1 > \dots > \sigma_m \geq 0$ and $n \geq m$. Using the Frobenius norm,

$$\|X - WX\|_F^2 = \|U\Sigma V^\top - WU\Sigma V^\top\|_F^2 = \|\Sigma - (U^\top WU)\Sigma\|_F^2, \quad \|W\|_F^2 = \|U^\top WU\|_F^2.$$

With $A := U^\top WU$ (still $m \times m$), we have

$$\mathcal{L}_2(W; X, \lambda) = \|\Sigma - A\Sigma\|_F^2 + \lambda\|A\|_F^2 = \sum_{i=1}^m (\sigma_i - a_{ii}\sigma_i)^2 + \sum_{i \neq j} a_{ij}^2 \sigma_j^2 + \lambda \sum_{i,j} a_{ij}^2.$$

The terms a_{ij} (for $i \neq j$) appear only quadratically with positive coefficients $\sigma_j^2 + \lambda$, so they are minimal at $a_{ij} = 0$. Hence the optimizer is diagonal in the U -basis, $A = \text{diag}(a_1, \dots, a_m)$, and we can write

$$\min_{a_i \in \mathbb{R}} \sigma_i^2(1 - a_i)^2 + \lambda a_i^2 \implies 2\sigma_i^2(a_i - 1) + 2\lambda a_i = 0 \implies a_i = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}.$$

Therefore the optimal encoder has the form

$$\widehat{W} = U \text{diag}\left(\frac{\sigma_1^2}{\sigma_1^2 + \lambda}, \frac{\sigma_2^2}{\sigma_2^2 + \lambda}, \dots, \frac{\sigma_m^2}{\sigma_m^2 + \lambda}\right) U^\top.$$

(c)

We are given singular values

$$\{\sigma_i\}_{i=1}^8 = \{10, 8, 4, 1, 0.5, 0.36, 0.16, 0.01\}.$$

From part (b), the gain applied to singular vector i by $\widehat{W}$ is

$$g_i(\lambda) = \frac{\sigma_i^2}{\sigma_i^2 + \lambda}.$$

We want the top three directions preserved at least 80%, so

$$g_i(\lambda) \geq 0.8 \quad (i = 1, 2, 3) \iff \lambda \leq \left(\frac{1}{0.8} - 1\right) \sigma_i^2 = 0.25 \sigma_i^2.$$

We use the smallest σ_i^2 among the top three, so namely $\sigma_3 = 4$ so $\sigma_3^2 = 16$:

$$\lambda \leq 0.25 \times 16 = 4.$$

For the remaining directions ($i \geq 4$), we want at most 50%, and

$$g_i(\lambda) \leq 0.5 \iff \lambda \geq \left(\frac{1}{0.5} - 1\right) \sigma_i^2 = 1 \cdot \sigma_i^2.$$

To hold for all $i \geq 4$, take the largest σ_i^2 among those indices, which is $\sigma_4 = 1$ so $\sigma_4^2 = 1$, so

$$\lambda \geq 1.$$

Combining,

$$\boxed{1 \leq \lambda \leq 4}.$$

Any λ in this interval guarantees $\geq 80\%$ preservation of the first three singular directions and $\leq 50\%$ on the remaining ones.

Problem 3

(a)

- Each filter has shape $3 \times 3 \times 3$ (as it spans all 3 input channels)
- There are 4 filters (one per output channel)

So, the total number of learnable parameters is

$$3 \times 3 \times 3 \times 4 = \boxed{108}$$

(b)

We have two steps

- One 3×3 filter per input channel (no mixing of channels)
- Input has 3 channels

So the number of parameters is

$$3 \times (3 \times 3) = 27$$

- 1x1 convolution applied to all depthwise outputs

- Input has 3 depthwise output channels
- Output has 4 channels

Each 1×1 filter takes 3 input channels and produces one output channel.

$$1 \times 1 \text{ conv parameters} = 1 \times 1 \times 3 \times 4 = 12$$

$$\text{Total} = 27 + 12 = \boxed{39}$$

Problem 4

(a)

To compute $\mathbb{E}[q^T k]$, as $q^T k = \sum_{i=1}^d q_i k_i$. Since each q_i and k_i are iid, we have $\mathbb{E}[q^T k] = \sum_{i=1}^d \mathbb{E}[q_i] \mathbb{E}[k_i] = \sum_{i=1}^d \mu^2 = d\mu^2 = \|\mu\|_2^2$. Thus, $\mathbb{E}[q^T k]$ depends linearly on both d and μ .

(b)

When $\mu = 0$ and $\sigma = 1$, we can find $\text{Var}(q^T k)$ by expanding $\text{Var}(q^T k) = \mathbb{E}[(q^T k)^2] - (\mathbb{E}[q^T k])^2$. Since $\mathbb{E}[q^T k] = 0$, this reduces to $\text{Var}(q^T k) = \mathbb{E}[(q^T k)^2]$. Each term in the dot product contributes equally and independently, so $\text{Var}(q^T k) = d \text{Var}(q_1 k_1) = d\sigma^4 = d$, and the variance of the dot product grows linearly with d , as desired.

(c)

Continuing with $\mu = 0$ and $\sigma = 1$, let s be the scaling factor so that $\mathbb{E}\left[\frac{q^T k}{s}\right] = 0$ and $\text{Var}\left(\frac{q^T k}{s}\right) = 1$. From part (b), we have $\text{Var}(q^T k) = d$, so $\text{Var}\left(\frac{q^T k}{s}\right) = \frac{d}{s^2}$. Setting this equal to 1 gives $s = \sqrt{d}$. Therefore, dividing the dot product by $\sqrt{d}$ normalizes its variance, which motivates the scaling used in the attention mechanism.

Problem 5

(a)

From Equation (8),

$$V'_i = \frac{\sum_{j=1}^N \text{sim}(Q_i, K_j) V_j}{\sum_{j=1}^N \text{sim}(Q_i, K_j)}.$$

For V'_i to be well-defined for all i , the denominator must never vanish. So,

$$\sum_{j=1}^N \text{sim}(Q_i, K_j) \neq 0 \quad \text{for all possible } Q, K.$$

A condition is that $\text{sim}(q, k)$ always has the same sign. We restrict attention to kernel similarities

$$k(x, y) : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}_+,$$

which guarantees positivity.

(b)

(i)

For a polynomial kernel of degree 2,

$$\text{sim}(q, k) = (q^\top k + 1)^2.$$

This can be written as the following:

$$\text{sim}(q, k) = \phi(q)^\top \phi(k).$$

(ii)

An explicit feature map $\phi : \mathbb{R}^D \rightarrow \mathbb{R}^C$ with $(q^\top k + 1)^2$ is

$$\phi(x) = [1, \sqrt{2}x_1, \dots, \sqrt{2}x_D, x_1^2, \dots, x_D^2, \sqrt{2}x_1x_2, \dots, \sqrt{2}x_{D-1}x_D]^\top,$$

where $C = \Theta(D^2)$.

(iii)

If the similarity has the form

$$\mathcal{K}(q, k) = \phi(q)^\top \phi(k),$$

then Equation (8) becomes

$$V'_i = \frac{\sum_{j=1}^N \phi(Q_i)^\top \phi(K_j) V_j}{\sum_{j=1}^N \phi(Q_i)^\top \phi(K_j)}.$$

(c)

Using the direct implementation of Equation (9), computing all pairwise scores $QK^\top$ is $O(N^2D)$, normalization via softmax over each row costs $O(N^2)$, multiplying attention weights with V costs $O(N^2M)$.

So, the total time complexity is

$$O(N^2(D + M)).$$

For memory, storing $Q, K \in \mathbb{R}^{N \times D}$ requires $O(ND)$, $V \in \mathbb{R}^{N \times M}$ requires $O(NM)$, and the attention matrix requires $O(N^2)$. So,

$$\text{Space} = O(N(N + D + M)) = O(N \max\{N, D, M\}).$$

(d)

Using associativity, Equation (10) can be written as

$$V'_i = \frac{\phi(Q_i)^\top \left(\sum_{j=1}^N \phi(K_j) V_j^\top \right)}{\phi(Q_i)^\top \left(\sum_{j=1}^N \phi(K_j) \right)}.$$

Let $C = \dim(\phi) = \Theta(D^2)$ and assume $M \approx D$.

For time complexity, forming $\sum_j \phi(K_j) V_j^\top$ is $O(NCM)$, forming $\sum_j \phi(K_j)$ is $O(NC)$, and computing all outputs is $O(NCM)$. So, the total time is $O(NCM) = O(ND^3)$. When $N \gg D^2$, this is cheaper than $O(N^2D)$.

Storing Q, K costs $O(ND)$, V, O cost $O(NM)$, and the accumulated feature matrices cost $O(CM) + O(C)$. Thus,

$$\text{Space} = O(ND + CM) = O(ND + D^3).$$

For $N \gg D^2$, kernelized attention uses much less memory than softmax attention.

Problem 6

(a)

Let $m = \frac{1}{n} \sum_{i=1}^n x_i$. When a new point x_{n+1} arrives the updated mean is $m' = \frac{1}{n+1} \sum_{i=1}^{n+1} x_i$. Writing this in terms of m we get $m' = \frac{n}{n+1}m + \frac{1}{n+1}x_{n+1}$, or $m' = m + \frac{1}{n+1}(x_{n+1} - m)$.

(b)

For data vectors $x_i \in \mathbb{R}^d$ and scalars y_i the ridge solution is $w^* = (X^\top X + \lambda I)^{-1} X^\top y$, where $X = [x_1, \dots, x_n]^\top$ and $y = [y_1, \dots, y_n]^\top$. To compute w^* we can get $X^\top X = \sum_{i=1}^n x_i x_i^\top$ and $X^\top y = \sum_{i=1}^n x_i y_i$. Each new point updates these two sums by one outer product and one scaled vector, and then a single matrix solve with $(X^\top X + \lambda I)$ gives w^* .

(c)

Treating d as the dimension of key/query/value vectors and n as the number of positions, the dominant work is forming and inverting the $d \times d$ matrix $A^\top A + \lambda I$ where A stacks the n key vectors, and then multiplying by the $n \times d$ value matrix. Forming $A^\top A$ is $O(nd^2)$, inverting the $d \times d$ matrix is $O(d^3)$. As the $O(nd^2)$ term dominates, and applying the resulting transform to all queries/values is another $O(nd^2)$, the overall complexity is $O(nd^2)$.

(d)

The ridge operation is formed from differentiable algebraic operations (matrix multiplies, additions, and an inverse). So, gradients flow through the entire computation. The learnable ridge coefficient λ receives gradients, and the key, value and query embeddings all influence the final outputs and thus receive gradients during backpropagation.

(f)

To compute causal ridge attention for time steps $t = 1, \dots, n$ without recomputing from scratch at each t , we can have running variables $S_t = \sum_{i=1}^t k_i k_i^\top$ and $r_t = \sum_{i=1}^t k_i v_i^\top$ (where k_i are keys and v_i values) together with the inverse $M_t^{-1} = (\lambda I + S_t)^{-1}$. When a new key-value (k_{t+1}, v_{t+1}) arrives update $S_{t+1} = S_t + k_{t+1} k_{t+1}^\top$ and $r_{t+1} = r_t + k_{t+1} v_{t+1}^\top$ and obtain M_{t+1}^{-1} in $O(d^2)$. Then, the transform needed for the current query is $W_{t+1}^\top q = r_{t+1}^\top M_{t+1}^{-1} q$ (or $(M_{t+1}^{-1} r_{t+1})^\top q$), which is computable with $O(d^2)$ work per step. So, by re-using M_t^{-1} and r_t we have a solution that is $O(d^2)$ per timestep rather than $O(td^2)$.

(g)

For a fixed query q the weights assigned to the n value vectors are the components of the vector $w = B^\top (A(A^\top A + \lambda I)^{-1} q)$, where $A = [k_1, \dots, k_n]^\top$ contains the keys and $B = [v_1, \dots, v_n]^\top$ the values. The i -th scalar weight equals $v_i^\top (A(A^\top A + \lambda I)^{-1} q)$ and, if we ignore the values, the normalized contribution of key k_i to the attention is governed by the inner product $k_i^\top (A^\top A + \lambda I)^{-1} q$. This shows ridge-attention computes inner products against a transformer query and does not enforce positivity or a softmax normalization, so weights can be negative and their scale depends on λ and the global key covariance.

(h)

Embedding absolute position information into the keys/queries themselves is straightforward to combine with ridge-attention because the formulas remain valid once keys and queries are shifted. Contrastingly, learned relative position embeddings that change pairwise interactions or require reparameterizing the $n \times n$ compatibility matrix are harder to reconcile with the rank-one update machinery and the fixed $d \times d$ linear algebra used in ridge updates. Therefore RoPE (because it embeds absolute position directly into key/query vectors) is the easier choice to adapt while preserving the structure used for efficient updates.

Problem 9

(a)

Vision transformer. The vision transformer processes the entire image at once and produces a set of latent image representations, and there is no notion of sequential generation.

Language transformer. Caption generation is sequential, as each token must be generated conditioned only on previously generated tokens and the image representation. This requires autoregressive self-attention masking and cross-attention over the visual features.

(b)

(i) Each column corresponds to a query, while each row corresponds to a key-value pair. The columns represent language tokens being generated by the decoder, which issue queries. The rows include both language tokens and image tokens, which act as keys and values that the decoder attends to through self-attention and cross-attention.

(c)

An image of size $H \times W$ is divided into patches of size $P \times P$, and there are $N = \frac{H}{P} \cdot \frac{W}{P}$ patch tokens. The attention runtime is $O(N^2 D) = O\left(\left(\frac{HW}{P^2}\right)^2 D\right)$, where D is the embedding dimension.

(d)

Each window has K^2 tokens, and there are $\frac{HW}{P^2 K^2}$ windows. Self-attention within a single window is $O(K^4 D)$, so the total attention over the entire image is $O\left(\frac{HW}{P^2 K^2} \cdot K^4 D\right) = O\left(\frac{HW}{P^2} K^2 D\right)$.